

Verifiable Governance for Agent Assisted Software Development

Separated roles, evidence contracts and context engineering

Andrés Felipe Reyes Gallego

Technical design paper · September 10, 2026 · English edition 1.1

Abstract. This paper presents a governance design for the software development life cycle supported by AI agents. The framework structures work through separated roles, explicit handoffs, configurable acceptance gates and claim verification against referenced artifacts. Two design documents and the agentic-sdlc-framework 1.7.0 package are examined. Static inspection identifies concrete validation mechanisms alongside gaps between declared controls and their enforcement. No experimental gains in productivity, accuracy or cost are reported. The contribution is a traceable architecture and an evaluation agenda for testing its benefits before attributing them to operational use.

Index terms: agentic SDLC, governance, evidence, adversarial verification, context engineering, SRE.

1 Introduction

When an agent interprets a request, produces code and evaluates its own delivery, a convincing conclusion can be difficult to distinguish from independent verification. The distinction becomes operational when that conclusion authorizes a change: the decision owner, evaluated version and supporting evidence must be identifiable.

The proposed framework distributes these responsibilities and makes handoff conditions explicit. Its objective is to let a team reconstruct delivery decisions and identify missing evidence before advancing. This is a process design objective; its impact on defects, lead time and cost requires measurement.

Anthropic distinguishes workflows with predefined paths from agents that dynamically direct their execution [4]. This framework is closer to the former: it defines a work sequence and routing rules, although agents may perform the activities within each step.

2 Materials and review scope

The review covers Fundamentos para trabajar con agentes, a 12-page source document [1]; the 28-page Framework SDLC Agéntico guide, whose headers identify v1.4.0 [2]; and the archive distributed as agentic-sdlc-framework-v1.7.0, whose VERSION file declares 1.7.0 [3]. Package code takes precedence when describing implemented behavior; the guide explains design intent.

The method is document review and static inspection of contracts, scripts and role definitions. No agents,

deployments or comparative trials were executed for this review. The framework evidence/ and metrics directories, named evidencia/ and metricas/ in the package, contain no result files. JSON files under ejemplos/ illustrate contracts and are not evidence of effectiveness.

3 Architecture and separation of authority

The design distinguishes seven delivery roles: technical lead, architect, builder, testing, security, SRE and DevOps. An eighth role, the verifier, evaluates producer claims according to impact and configuration. Routes vary by work type; an investigation ends in a document and requires a new work order before implementation [2, 3].

The technical lead sets scope and resolves blockers. Architecture defines contracts and impact; the builder implements; testing and security produce evidence and assessments. SRE specifies service objectives, observability and rollback. DevOps prepares and executes an authorized delivery. Blocked work returns to the technical lead for scope or assignment decisions.

Reuse depends on separating role identity, procedure and acceptance criteria. Procedures reside in skills; thresholds and activation conditions are declared in contratos/gates.yaml; routing is defined in contratos/roles.yaml. A project constitution records overarching principles and their associated gates. A coverage threshold should be adjustable without rewriting the testing procedure.

Documented separation requires effective environmental controls. A written prohibition on editing code is not a read-only permission. The inspected verifier definition includes Bash and textual restrictions; it does not by itself establish write isolation. Adoption must verify process permissions, tool access and the evidence boundary.

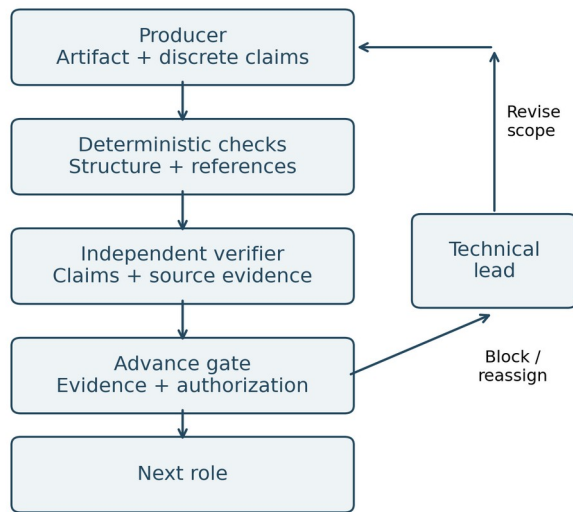


Figure 1. Intended evidence flow when verification is active. Gate enforcement and permission isolation require implementation and validation; the diagram is a design view.

4 Claims and verifiable evidence

Each producer can emit discrete claims with an identifier, type, reference and criticality. The verifier receives those claims and relevant artifacts without the producer's explanatory rationale. The specified verdicts are VERIFICADO, CORREGIDO and RECHAZADO, corresponding to verified, corrected and rejected. Context separation aims to reduce confirmation of earlier conclusions, but agents sharing a model can retain correlated errors.

The design acknowledges Cloudflare's security audit pattern with machine-readable findings and independent review [6]. Its documented adaptation extends the structure to other delivery roles. Spec Kit is acknowledged as an influence on the project constitution [7]. These relationships explain the framework's composition; they do not establish scientific priority for every component.

resolver-referencias.sh checks file existence and line ranges and extracts fragments for review. A resolving reference does not prove that the source supports the claim: that relationship requires contextual inspection or execution. Checking command availability likewise does not establish successful execution.

validar-salida.sh implements a subset of structural validation and semantic rules, including rejecting blocking claims marked no_verificable. It detects certain format defects; it should not be described as a complete JSON Schema implementation or proof of functional correctness.

5 Observed state and gate limitations

Inspection of gates.yaml identifies 23 declared gates, including 7 marked verificacion: manual. Some guide passages refer to four manual gates. This discrepancy should be reconciled before treating the guide and package

as a single baseline [2, 3]. Figure 2 reports the actual declaration counts, not successful executions.

Element	Observation in 1.7.0
Gate catalog	23 declared; 7 manual
Advance validation	Reads handoff states
References	Checks files and lines
Empirical results	Not included in package

Table 1. Static inspection findings, not execution results.

verificar-gates.sh determines applicability and reads states from the latest handoff. In its aprobado branch, it prints the state and declared evidence. It does not generally execute the hash, coverage or file-presence checks described in the YAML. An approved state alone therefore does not establish that those checks occurred.

The proposed next step is to map each verification type to a deterministic evaluator, validate exceptions and expiry dates, and bind evidence to the work order and artifact hash. Missing or inconsistent required evidence should block advancement. This is a proposed improvement, not a certified property of the reviewed release.

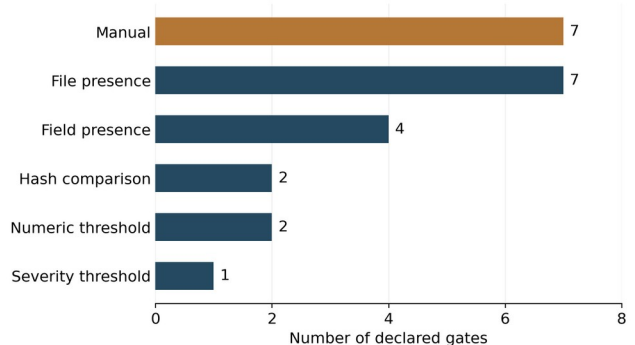


Figure 2. Declared verification types in contratos/gates.yaml, version 1.7.0 (n = 23). Counts were extracted from the package. Non-manual declarations do not imply implemented evaluators.

6 Context engineering and cost

The foundations distinguish instruction writing, context selection and system design [1]. The framework organizes context into stable information, working material and the current task. It proposes managing relevance, freshness, retrieval and placement instead of loading documents and history indiscriminately.

Liu and colleagues observed that information placement affected performance in the tasks and models they studied [5]. This supports evaluating context placement; it does not establish universal percentages or guarantee an optimal layout for every model. Unverified numerical claims from the source training material are therefore omitted.

The package records input, output and cached tokens, duration and model. Its report aggregates consumption and verdict distribution; it does not calculate complete

monetary cost. It also does not measure energy. Lower token use alone cannot establish lower energy consumption without information about computation, hardware and execution conditions.

7 Proposed evaluation protocol

Three conditions should be compared on the same tasks: a single agent with usual human review; a role chain with deterministic checks and no adversarial verifier; and the same chain with an isolated verifier. All conditions share acceptance tests. Comparing the latter two estimates the additional effect of verification.

Tasks should cover code, contracts and infrastructure, with acceptance criteria defined before execution. Each condition uses clean sessions and records model version, tools, configuration, repository version and budget. Execution order should be alternated and tasks repeated to observe variability. Human assessment should conceal the condition where feasible.

The primary outcome is the proportion of deliveries containing relevant undetected defects, adjudicated through tests and independent review. Additional measures include false rejections, total time, rework, human review minutes and cost per accepted delivery. Verifier finding rate must be interpreted alongside missed defects: more findings do not necessarily imply better final quality.

Proposed total cost includes inference, tools, infrastructure and human work. Cost per accepted delivery divides the cost of all attempts, including failed ones, by accepted deliveries only. Sample sizes, dispersion and differences by impact must be reported. The task count should be determined after a pilot rather than selected to support a desired improvement.

8 Conclusion

The framework provides a concrete structure for assigning responsibilities and connecting claims to artifacts. Its potential value lies in making delivery decisions inspectable. The review also identifies controls that remain declarative, inconsistent documentation versions and an absence of operational results in the supplied package.

The next contribution should be experimental: complete gate evaluators, demonstrate verifier isolation and measure quality and cost against a comparable baseline. Until then, the work can be shared as a documented technical design with explicit limitations and without unsupported accuracy or savings claims.

References

- [1] Fundamentos para trabajar con agentes. Author-supplied source document, 12 pages, especially sections 4, 10 and 15. In Spanish.
- [2] Framework SDLC Agéntico guide. Author-supplied source document, 28 pages, headers v1.4.0; sections 2–6 and 12–14. In Spanish.
- [3] agentic-sdlc-framework 1.7.0. Author-supplied package. VERSION, contratos/gates.yaml, contratos/roles.yaml, validation scripts and .claude/agents/verificador.md. Inspected September 10, 2026.
- [4] Anthropic. Building effective agents. December 19, 2024. [Source](#)
- [5] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, vol. 12, pp. 157–173, 2024. arXiv:2307.03172. [Source](#)
- [6] Cloudflare. security-audit-skill. Public repository, accessed September 10, 2026. [Source](#)
- [7] GitHub. Spec Kit. Public repository, accessed September 10, 2026. [Source](#)

Editorial note. Technical design and document review; no empirical evaluation or peer-review claim. AI-assisted writing based on the supplied research materials.